

جامعة خليفة
Khalifa University

Department of Computer Science
Computer System Organizations
Spring 2025

Project Phase 3:
Logic Bomb

Prepared by:

Tehsin Shaik 100064806
Raghad Elajab 100064670

Submitted to:
Dr. Azzam Mourad

Submission Date:
27/04/2025

Registered Bomb Number:
bomb29

Introduction

This report outlines our approach to defusing the binary bomb assigned to our team as part of the COSC201: Computer System Organization course project. The binary bomb is a compiled executable containing multiple phases, each requiring a precise input string to proceed. Any incorrect input results in the bomb "exploding".

To begin, we extracted the bomb's full assembly code using objdump and then disassembled code by grouping each phase with its associated helper functions, which allowed us to better understand the control flow and logic behind each input check.

Throughout the project, we consistently used the GDB, placing breakpoints at both explode_bomb and the specific phase we were working on. In some cases, we also added breakpoints inside functions called within the phases to better understand what inputs or conditions would trigger the bomb.

This report provides a detailed breakdown of the steps taken to analyze and defuse each phase, along with screenshots and explanations to illustrate our process and findings.

Phase 1:

To begin defusing Phase 1, we analyzed the assembly code for phase_1 and noticed it was calling a function (called strings_not_equal) which compares two strings, and one of the inputs was a value from a specific memory address.

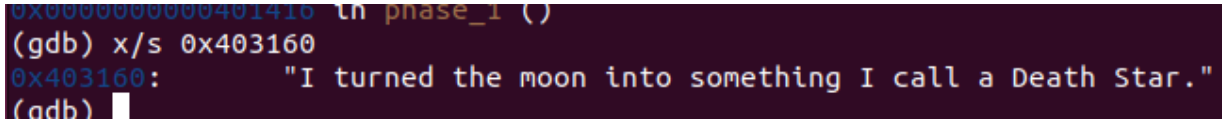
Just before this function call, we observed the instruction:

```
0x401412 <phase_1>      sub    $0x8,%rsp
0x401416 <phase_1+4>    mov    $0x403160,%esi
0x40141b <phase_1+9>    call   0x4018e2 <strings_not_equal>
0x401420 <phase_1+14>   test   %eax,%eax
```

Caption: Assembly code of phase_1 showing the hardcoded string address and the call to strings_not_equal.

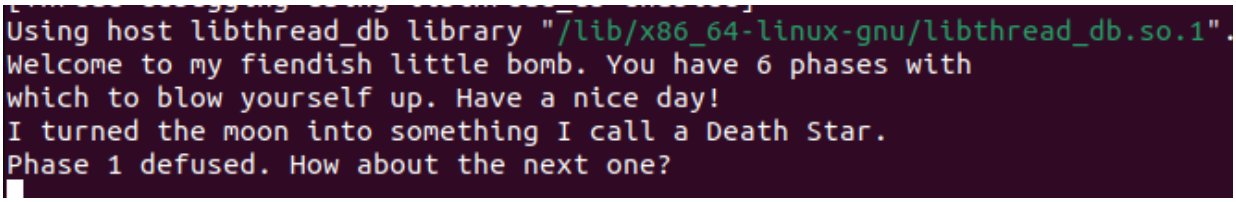
This indicated that one of the arguments being passed to strings_not_equal is a string stored at the memory address 0x403160. Since the function is checking whether the user input matches this hardcoded string, we concluded that retrieving the contents at this address would likely reveal the correct input required to defuse the phase.

Using gdb, we examined the memory at this address with the following command:



Caption: Reading memory at address 0x403160 to reveal the correct input string.

We tested this by entering the string when prompted, and the bomb proceeded to the next phase without exploding. This confirmed our analysis was correct.



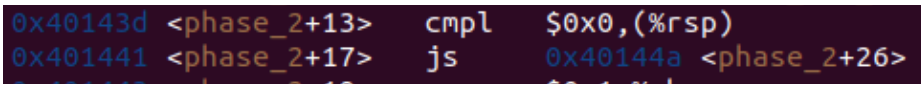
Caption: Correct input entered—Phase 1 defused successfully.

So the final answer was “I turned the moon into something I call a Death Star.”

Phase 2:

For Phase 2, we began by analyzing the disassembled code in bomb.s and immediately noticed that the function phase_2 begins with a call to read_six_numbers. This helper function reads six integers from the user and stores them on the stack. We confirmed this by placing breakpoints in GDB at both phase_2, read_six_numbers and explode_bomb, allowing us to step through how the input was handled.

The first validation check appears here. This comparison checks if the first input number is less than zero. If it is, execution jumps to explode_bomb. From this, we determined that the first number must be a non-negative number.



Caption: Assembly code showing the comparison logic.

Next, we analyzed the loop that follows. The loop uses the %ebx register as an index (i), starting from 1 and incrementing it, up to 5. Within the loop, the following key instructions perform a check:

```
   0x40145b <phase_2+43>    mov    %ebx,%eax
>  0x40145d <phase_2+45>    add    -0x4(%rsp,%rbx,4),%eax
   0x401461 <phase_2+49>    cmp    %eax,(%rsp,%rbx,4)
   0x401464 <phase_2+52>    je     0x401451 <phase_2+33>
   0x401466 <phase_2+54>    call   0x401afd <explode_bomb>
   0x40146b <phase_2+59>    jmp    0x401451 <phase_2+33>
```

Caption: Assembly code showing the main pattern logic.

We understood that in each iteration, it adds the previous number numbers[i-1] to the current index i, and compares the result sum to the current number numbers[i]. If the values do not match, it calls explode_bomb.

Here, %rbx holds the current index i, -0x4(%rsp,%rbx,4) refers to the previous number numbers[i-1], the previous number on the stack. This value is added to %eax, which is initialized with the value of %ebx (i.e., the current index i). The result in %eax becomes numbers[i-1]+i. That sum is compared to numbers[i], which is at memory location (%rsp,%rbx,4).

We tested this in GDB by entering dummy input like 1 2 3 4 5 6, and then observed how %eax, the previous number, and the current value were compared.

```
(gdb)
Phase 1 defused. How about the next one?
1 2 3 4 5 6
```

Caption: Testing with different inputs for defusing Phase 2

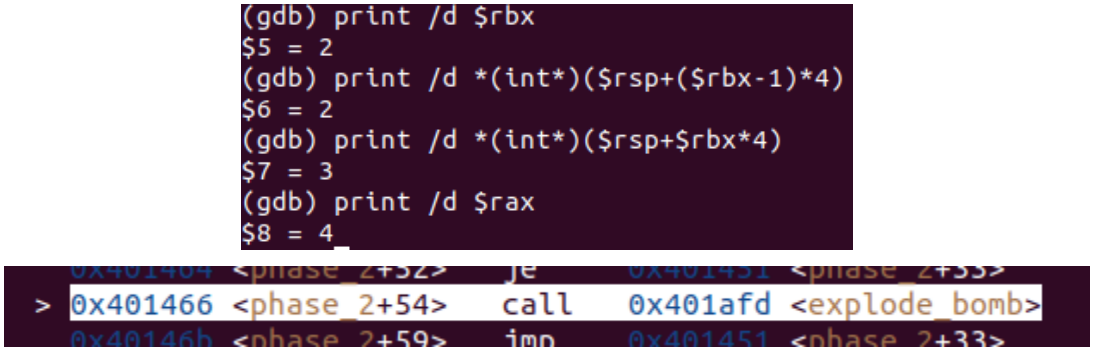
Using GDB commands like print /d to inspect the register and memory values, we confirmed the rule that numbers[i] == numbers[i-1] + i.

For input like 1 2 3 4 5 6, the condition holds in the first iteration.

```
(gdb) print /d $rbx
$1 = 1
(gdb) print /d *(int*)($rsp+($rbx-1)*4)
$2 = 1
(gdb) print /d *(int*)($rsp+$rbx*4)
$3 = 2
(gdb) print /d $eax
$4 = 2
```

Caption: GDB output during Phase 2 showing numbers[1] == numbers[0] + 1 → 2 == 1 + 1

But in the second iteration, at index i = 2, the expected value is 4, but the actual input is 3, causing the bomb to explode.



Caption: GDB output during Phase 2 showing failure of the rule.
numbers[2] == numbers[1] + 2 → 3 != 2 + 2

Once we figured that logic out, we selected 1 as a valid starting number (since it is non-negative and didn't explode) and generated the rest accordingly. That gave us:

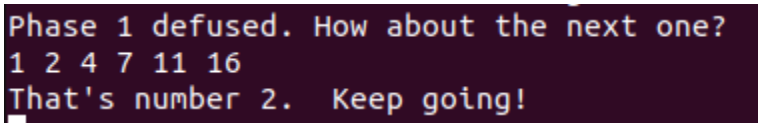
1 (initial)
2 = 1 + 1
4 = 2 + 2
7 = 4 + 3
11 = 7 + 4
16 = 11 + 5

Final input: 1 2 4 7 11 16

We entered this into the bomb and confirmed in GDB that the loop completed successfully and returned without calling explode_bomb.

To verify our understanding, we also tested several random inputs. Any deviation from the pattern rule caused the program to jump to explode_bomb, confirming that the rule was strictly enforced.

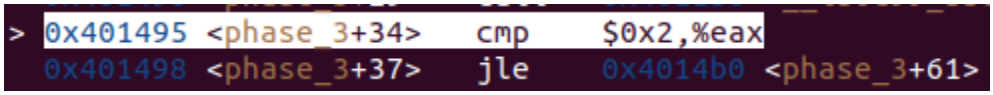
So for Phase 2, the final answer was: "1 2 4 7 11 16"



Caption: Correct input entered—Phase 2 defused successfully.

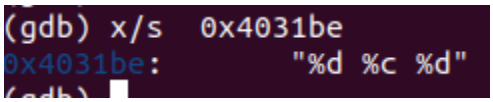
Phase 3:

For Phase 3, we began by examining the phase_3 function in the disassembled bomb code. We saw that the code is expecting three user inputs. The function starts by calling sscanf to parse the input and checks whether it successfully read at least two values. If it didn't, the bomb immediately explodes, so right away, we knew we had to input three valid values in the correct format.



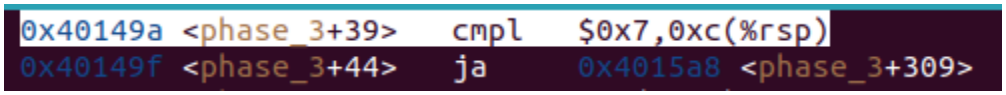
Caption: the compare call that would jump to explode_bomb if we input less than 3 inputs

We used x/s 0x4031be because that address was moved into %esi right before calling sscanf, meaning it must point to the format string used for parsing input. From this, we understood that we needed an integer, a character, and another integer.



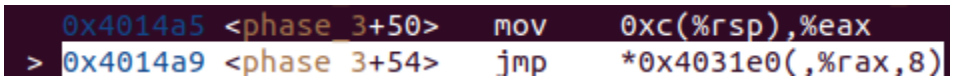
Caption: Inspecting the format string at address 0x4031be, which is passed to sscanf

After validating the input format, the function checks if the first integer (stored at 0xc(%rsp)) is greater than 7. This tells us the first integer input must be in the range 0 to 7. If it's not, the bomb explodes.



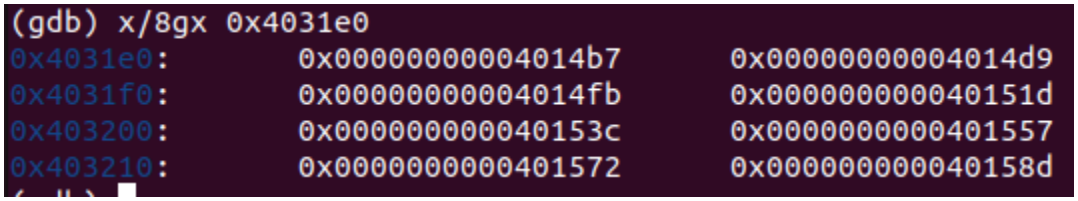
Caption: Checking if the first input number is greater than 7

The first integer acts as a sort of selector. It is used as an index into a jump table at memory address 0x4031e0 that controls which code path the function follows. Each path (case 0 to case 7) has its own expected character and integer. The function uses this index to jump to a specific code block, and then checks if the character and second number match exactly what it's looking for. If either value is wrong, it jumps to explode_bomb. Only the exact combination of all three inputs will safely pass the phase.



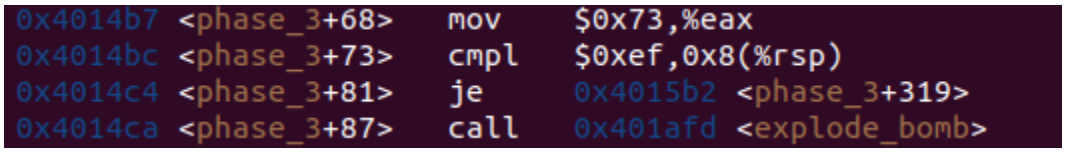
Caption: Jumping to the appropriate case in the jump table based on the first input value.

We used the command x/8gx 0x4031e0 to inspect 8 entries of the jump table. Each entry was an address corresponding to a case block.



Caption: Viewing the jump table at address 0x4031e0, which holds pointers to the start of each case block indexed by the first input.

We used GDB to set breakpoints and jump to each of these addresses to inspect what each case checked for. For example, case 0 expects the character s and the number 239, case 2 wants u and 532, and so on. We understood this as 0x73 in ASCII is the letter 's', so this tells us case 0 expects the character input to be 's'. 0xef is 239 in decimal, so the third number must be 239.



Eventually, we found that case 1 was looking for the character g and the number 543. So, as long as the first input is 1, the second is g, and the third is 543, the function passes without blowing up the bomb.

While initially we assumed there might be only one correct combination of inputs. However, after further experimentation and analysis, we realized that each case in the jump table has its own valid combination of values and any of them will successfully pass the phase if matched exactly. So, if you provide a valid index (0-7) and the correct character and number expected for that case, then the bomb won't explode.

Table: Valid input combinations for Phase 3 — each row represents a correct triplet of values (index, character, number) that will defuse the bomb when entered in the expected format.

input 1 (%d)	input 2 (%s)	input 3 (%d)
0	0x73 (s)	239
1	0x67 (g)	543
2	0x75 (u)	532

3	0x6c (l)	154
4	0x64 (d)	633
5	0x72 (r)	601
6	0x74 (t)	650
7	0x68 (h)	247

We tested this by inputting 1 g 543, stepped through it in gdb, and confirmed it matched the expected values for case 1 and avoided any bomb-triggering branches.

So for Phase 3, the final solution we chose was “1 g 543”

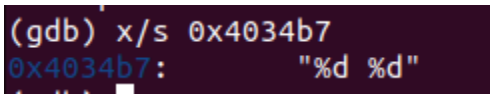


Caption: Correct input entered—Phase 3 defused successfully.

Phase 4:

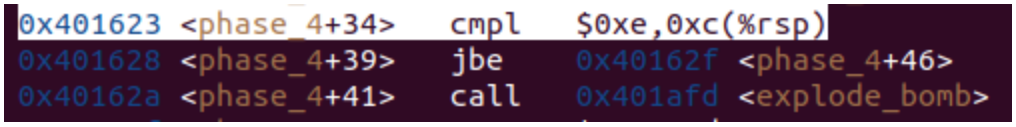
This phase involves a recursive function that resembles a binary search, and the user must input two integers that satisfy certain conditions checked by the program.

We started by examining the phase_4 function and noticed the call to sscanf, just like in earlier phases. To inspect the format string, we used the command and confirmed that the input format is two integers. If exactly two integers aren't parsed, the bomb explodes. So we knew the input needed to look like "%d %d", indicating that the function expects two integers as input.



Caption: Inspecting the format string used by sscanf at address 0x4034b7.

Next, we saw that there is a check that ensures the first number (stored at 0xc(%rsp)) is between 0 and 14, inclusive. This is done with a comparison to 0xe (which is 14 in decimal). If the number is greater than 14, the bomb explodes. So we knew the first number had to be somewhere between 0 and 14.



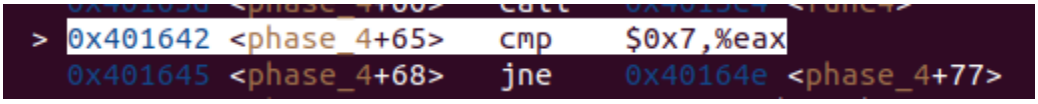
Caption: Check to ensure the first input is less than or equal to 14

Next, the program sets up arguments for a call to func4 passing in three parameters:

- edi (1st input from user): Target number to search for.
- esi = 0: Start of the search range.
- edx = 14: End of the search range.

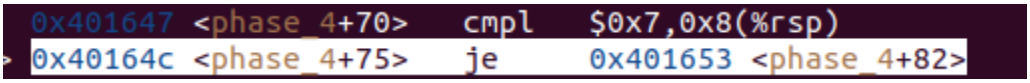
The func4 function turned out to be a recursive binary search. It calculates the midpoint and compares it to the target. If the target is less than the midpoint, it recurses on the left half and returns 2 * func4(...). If greater, it recurses on the right half and returns 2 * func4(...) + 1. If the midpoint equals the target, it returns 0.

Back in phase_4, the return value from func4 is compared against 7. So we needed to find a number between 0 and 14 where the binary search path encoding equals 7.



Caption: Comparison of the return value of func4 to 7

The second integer (stored at `0x8(%rsp)`) is then checked to see if it matches `7.` So, we knew that the second input was 7 for sure.



Caption: Comparison of 2nd input (at 0x8(%rsp)) to 7

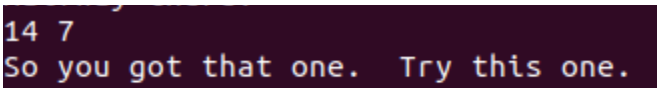
So, now, for the first input, we knew it had to be between 0 and 14, and that func4(first_input, 0, 14) had to return 7.

We realized the binary search was recursive, so we manually traced how it would behave for different input values. Then for each possible input from 0 to 14, we computed the recursive path and its encoded return value based on whether it moved left or right at each level. Though it took some time, we saw that when the target was 14.

Through this process, we discovered that 14 was the correct number. func4(14, 0, 14) returns 7, which passes the check. To verify this, we stepped through func4 in GDB with various inputs until we saw that when the first number is 14, func4 returns 7.

In GDB, we passed "**14 7**" and it passed through without exploding the bomb, and confirming that it worked.

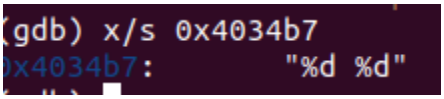
So for Phase 4, the final solution we chose was "**14 7**"



Caption: Correct input entered—Phase 4 defused successfully.

Phase 5:

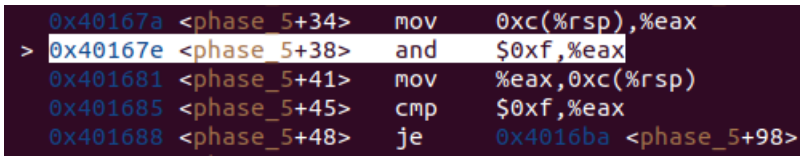
For Phase 5, we followed a similar approach as in previous phases to identify the number of inputs and their types. The format string at address 0x4034b7 is passed to sscanf to parse the user input. By inspecting this string in GDB, we determine the expected input format and validate the parsed values. The input format required two integers.



Caption: Inspecting the format string at address 0x4034b7, which is passed to sscanf

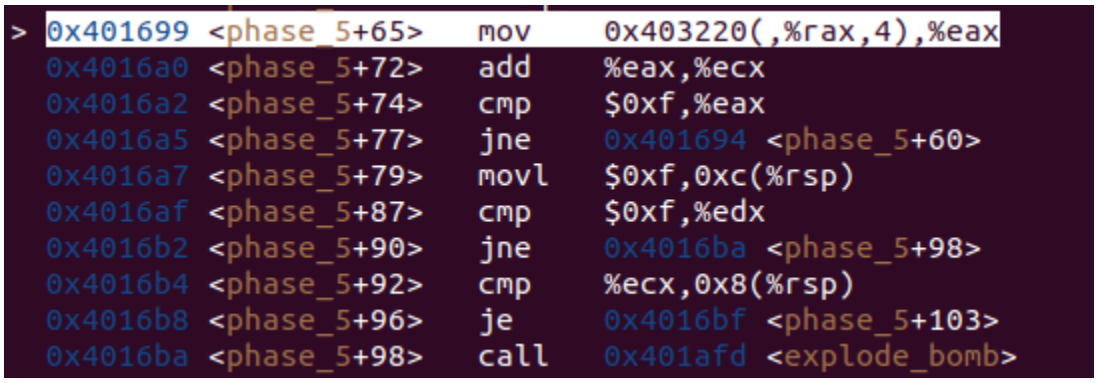
We then proceeded to analyze the assembly code and tried to convert it into C code for better clarity. Initially, we misinterpreted how the indexing into the table worked, assuming the values were just being summed directly. However, after looking closer, we identified that the values retrieved from the table were being used as indices to traverse through a sequence of values in the table rather than being summed directly.

Looking at the assembly code, we observed that the second input was masked with 0xF. This meant that the second input had to be in the range 0–15, as it would only use the last 4 bits. The masked value was used as the starting index to access the table, which is located at address 0x403220.



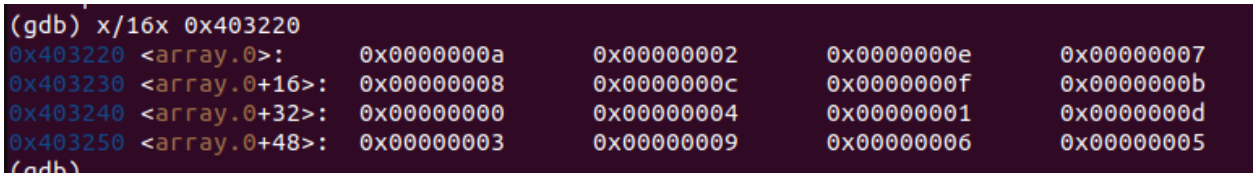
Caption: The second input masked with 0xF at address 0x40167a, indicating it must be between 0 and 15.

The table is iterated by following the values stored at each index, and the loop continues until the value 15 is encountered. Each value encountered in the table is added to a running total, starting at the index provided by the second input. The loop in the assembly code adds these values and stops when the value 15 is encountered. For the bomb to be defused, the chain must follow exactly 15 steps and the total sum must match the first input value.



Caption: The loop in the assembly iterates through the table values and accumulates the sum.

To understand the table values, we used the x/16x 0x403220 command in GDB to inspect the contents of the table. We printed out the values at address 0x403220 and the next 15 entries since we knew the table had 16 values. Then we tried different starting indices and followed the chain each time, checking how many steps it took and what the sum was.



Caption: The table values obtained by printing x/16x 0x403220 in GBD

After experimenting with different starting indices, we determined that starting at index 5 provided a 15-step chain that ended at 15 and resulted in a sum of 115. This allowed us to conclude that the correct input for Phase 5 was: " **5 115**"

```
5 115
Good work!  On to the next...
```

Caption: Correct input entered—Phase 5 defused successfully.

Phase 6:

At the beginning of phase_6, the program calls read_six_numbers, which uses sscanf with the format string "%d %d %d %d %d %d" to read six integers into a local array. If fewer than six numbers are read, the bomb explodes.

```
(gdb) x/s 0x4034ab
0x4034ab:       "%d %d %d %d %d %d"
```

*Caption**:** Format string passed to sscanf ensures input must contain six integers.*

Each number from the user input is first loaded and **decremented by 1**, effectively converting the range from **[1–6]** to **[0–5]** for internal indexing. The assembly code then checks if the result is greater than 5. If it is, the program jumps to the explode_bomb function. This means the original input values must be within the valid range of 1 to 6, inclusive. So, if any number is less than 1 or greater than 6, the bomb is triggered.

```
0x401714 <phase_6+73>   mov    %r12,%rbp
0x401717 <phase_6+76>   mov    (%r12),%eax
0x40171b <phase_6+80>   sub    $0x1,%eax
0x40171e <phase_6+83>   cmp    $0x5,%eax
0x401721 <phase_6+86>   ja     0x4016ec <phase_6+33>
```

At first, we noticed the phase accepts 6 numbers between 1 and 6 with no duplicates. It checks each number against all others to ensure they are unique. It also checks if they're in descending order. Naturally, we assumed it wanted 6 5 4 3 2 1 — but that triggered the bomb. So we knew something else was going on.

```
0x401700 <phase_6+53>   cmp    %eax,0x0(%rbp)
0x401703 <phase_6+56>   jne    0x4016f3 <phase_6+40>
0x401705 <phase_6+58>   call   0x401afd <explode_bomb>
```

We dug deeper and realized that those numbers weren’t being compared directly. Instead, each number pointed to a node in a hidden linked list. The program followed each number to fetch a node, and then stitched those nodes together in the same order as our inputs. Each node contains

a value, an index (from 1 to 6), and a pointer to the next node. The input numbers are used to select these nodes, and the program builds a new linked list in the order specified by the user's input. This part of the code starts at 0x401733, where it traverses a node list based on the user's choices, storing pointers in a local array.

```
0x401733 <phase_6+104>  mov    0x30(%rsp,%rsi,4),%ecx
0x401737 <phase_6+108>  mov    $0x1,%eax
0x40173c <phase_6+113>  mov    $0x4052d0,%edx
0x401741 <phase_6+118>  cmp    $0x1,%ecx
0x401744 <phase_6+121>  jle    0x401751 <phase_6+134>
0x401746 <phase_6+123>  mov    0x8(%rdx),%rdx
0x40174a <phase_6+127>  add    $0x1,%eax
0x40174d <phase_6+130>  cmp    %ecx,%eax
0x40174f <phase_6+132>  jne    0x401746 <phase_6+123>
0x401751 <phase_6+134>  mov    %rdx,(%rsp,%rsi,8)
```

After that, it compared the values stored inside those nodes, not the numbers we entered. That's when it clicked: it wanted the list to be sorted in descending order by node values, not by our inputs. It verifies that each node's value is greater than or equal to the next i.e., the list must be sorted in descending order. If at any point a node has a smaller value than the next, the bomb explodes.

```
0x4017a8 <phase_6+221>  mov    0x8(%rbx),%rax
0x4017ac <phase_6+225>  mov    (%rax),%eax
0x4017ae <phase_6+227>  cmp    %eax,(%rbx)
0x4017b0 <phase_6+229>  jge    0x40179f <phase_6+212>
0x4017b2 <phase_6+231>  call   0x401afd <explode_bomb>
```

So, we printed the values inside all six nodes and found their actual values. Once we knew which node had which value, we rearranged our input numbers to produce a list where the node values decreased. That input passed the phase.

```
Breakpoint 3 at 0x401afd
(gdb) x/24x 0x4052d0
0x4052d0 <node1>:       0x000001e1      0x00000001      0x004052e00x00000000
0x4052e0 <node2>:       0x0000023a      0x00000002      0x004052f00x00000000
0x4052f0 <node3>:       0x00000043      0x00000003      0x004053000x00000000
0x405300 <node4>:       0x0000012a      0x00000004      0x004053100x00000000
0x405310 <node5>:       0x0000018c      0x00000005      0x004053200x00000000
0x405320 <node6>:       0x00000296      0x00000006      0x000000000x00000000
```

Caption: The values inside the 6 nodes - they are in hex - as well as their corresponding node number

Sorting the values in descending order gave 662 (node 6), 570 (node 2), 481 (node 1), 396 (node 5), 298 (node 4), 67 (node 3). This way, the list satisfies all conditions: no duplicates, valid range, and correct descending order based on internal node values.

So for Phase 6, the final solution was: “6 2 1 5 4 3”

```
Good work!  On to the next...
6 2 1 5 4 3
Congratulations! You've defused the bomb!
Your instructor has been notified and will verify your solution.
```

Caption: Correct input entered—Phase 6 defused successfully.

Secret Phase:

After defusing all six regular phases of the binary bomb, the program didn’t automatically proceed to any additional logic, it just exited. We suspected there was a hidden “secret phase” and began investigating how it might be triggered.

We disassembled the phase_defused function using GDB and set breakpoints for the secret_phase, explode_bomb, and the function we found inside the secret_phase which is fun7. We noticed some interesting logic near the end. In particular, the function checks this.

401ca1: cmpl $0x6,0x3b68(%rip) # 405810 <num_input_strings>

This tells us it verifies that six inputs have been given (i.e., all six phases were completed).

Then, shortly after, **401cbe: mov $0x403501,%esi**
401cc3: mov $0x405910,%edi
401ccd: call __isoc99_sscanf@plt

This shows that sscanf is being used to parse more input, even after the bomb is defused. We wanted to see what format string was at 0x403501, since it was being moved to esi and used for sscanf so we used:

```
(gdb) x/8bs 0x403501
0x403501:       "%d %d %s"
0x40350a:       "DrEvil"
0x403511:       "cso-vm"
0x403518:       "cso-virtual-machine"
0x40352c:       "mario-virtual-machine"
0x403542:       ""
0x403543:       ""
0x403544:       ""
```

Caption: Reveals the format string used by sscanf

This revealed that the program is expecting two integers and a string. To see what values it uses for comparison, we looked at the memory at 0x405910 which is moved into edi before parsing.

```
(gdb) x/8bs 0x405910
0x405910 <input_strings+240>:   "14 7"
0x405915 <input_strings+245>:   ""
0x405916 <input_strings+246>:   ""
0x405917 <input_strings+247>:   ""
0x405918 <input_strings+248>:   ""
0x405919 <input_strings+249>:   ""
0x40591a <input_strings+250>:   ""
0x40591b <input_strings+251>:   ""
```

Caption: Reveals the format string stored inputs which sscanf will try to match

These matched our original input for Phase 4. So we guessed that the first two values expected here are the Phase 4 inputs.

Next, we traced the flow further. After sscanf, the code loads the string from 0x40350a and compares it using the strings_not_equal function. We dumped that memory too:

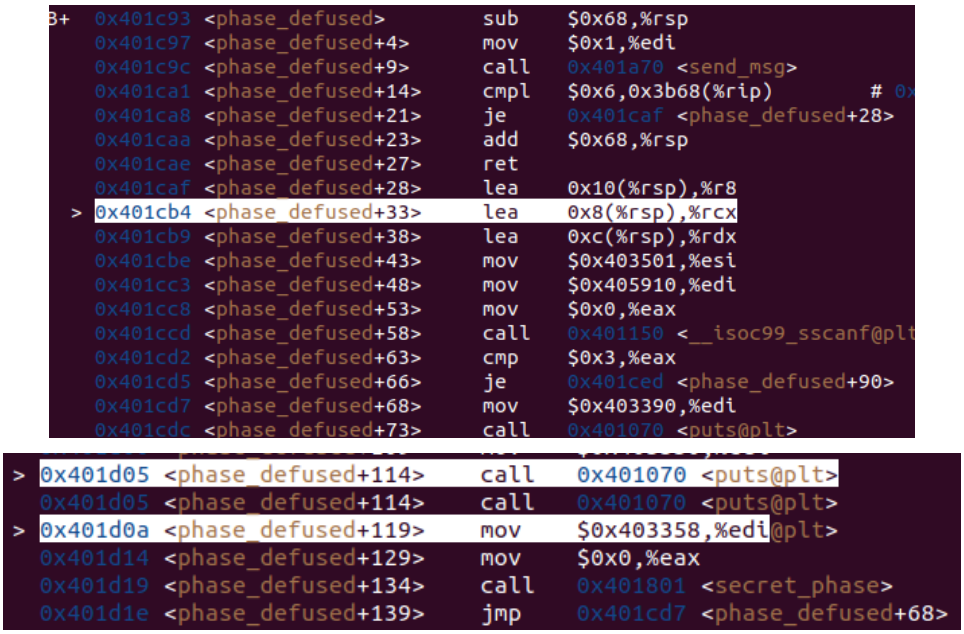
```
(gdb) x/8bs 0x40350a
0x40350a:       "DrEvil"
0x403511:       "cso-vm"
0x403518:       "cso-virtual-machine"
0x40352c:       "mario-virtual-machine"
0x403542:       ""
0x403543:       ""
0x403544:       ""
0x403545:       ""
```

Caption: Reveals the final string value "DrEvil" which is a secret keyword that must be entered after Phase 4 inputs to trigger the secret_phase

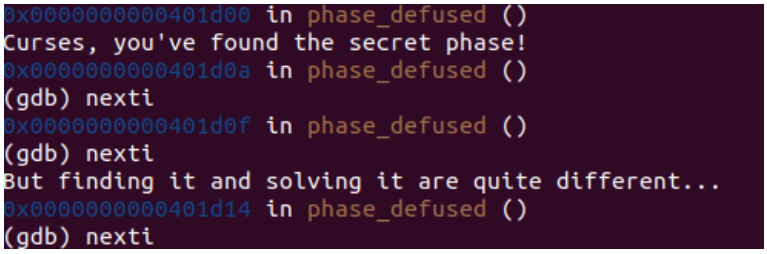
So, we found a string text called “DrEvil”. So now we had all three required pieces of input: 14, 7, and "DrEvil". We ran the phases as usual and in place for Phase 4’s input, we entered 14 7 DrEvil.This caused the program to proceed through the rest of the phases as usual.

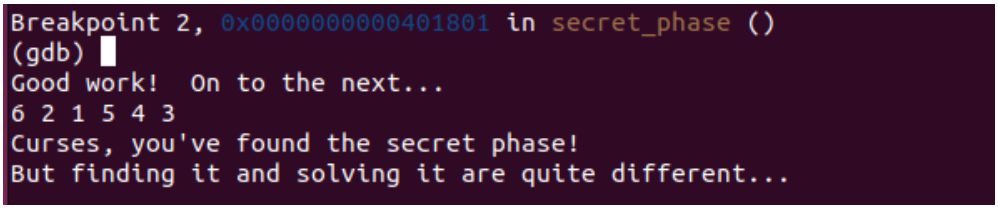
```
I turned the moon into something I call a Death Star.
Phase 1 defused. How about the next one?
1 2 4 7 11 16
That's number 2.  Keep going!
1 g 543
Halfway there!
14 7 DrEvil
```

However, after Phase 6, when control returned to phase_defused, this triggered a hidden code path that allowed access to the secret_phase function. So, instead of returning to main, it continued execution deeper into the function. We stepped through, and eventually reached the secret_phase.



We had officially entered the hidden Secret Phase.





Inside the secret_phase function, you are asked to enter a single value.The program reads the input and tries to turn it into an integer. If it's not a valid number or it's out of range (not between 1 and 1001 inclusive), the bomb explodes. We figured out that the input needed to be a valid number in this range.

```
0x401814 <secret_phase+19>      call   0x401130 <strtol@plt>
0x401819 <secret_phase+24>      mov    %eax,%ebx
0x40181b <secret_phase+26>      sub    $0x1,%eax
0x40181e <secret_phase+29>      cmp    $0x3e8,%eax
0x401823 <secret_phase+34>      ja     0x401846 <secret_phase+69>
```

Caption: The original input must be ≤ 1001 and ≥ 1.

Then, the valid number is then passed into fun7, along with a pointer to a binary search tree. Inside fun7, the number you input is used to go through a tree-like structure, with each step checking if the number should go to the left or right node, based on whether it's smaller or larger than the current number.

The number is compared with each node's value.

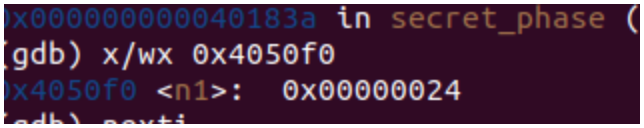
- If your number is less than the node's value, the function recursively goes left.
- If your number is greater, it goes right.
- If your number equals the node's value, it returns 0.

Each recursive call tracks the path taken through the tree. If the value is not found, or if it follows the wrong path, the function will return a non-zero result, causing the bomb to explode.

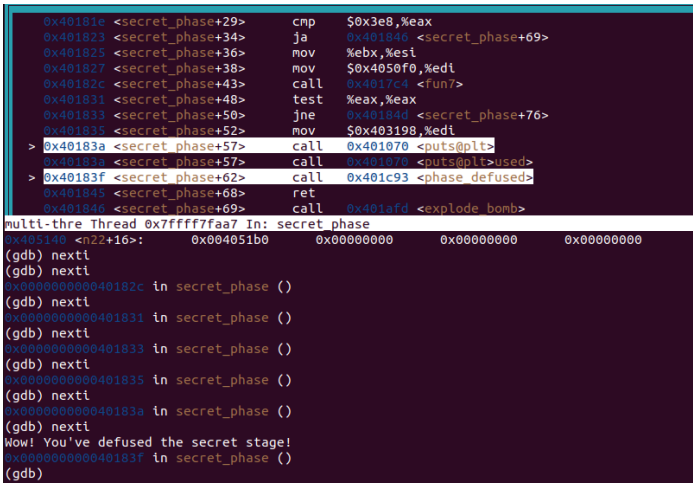
```
0x4017c4 <fun7>         test   %rdi,%rdi
0x4017c7 <fun7+3>       je     0x4017fb <fun7+55>
0x4017c9 <fun7+5>       sub    $0x8,%rsp
0x4017cd <fun7+9>       mov    (%rdi),%edx
0x4017cf <fun7+11>      cmp    %esi,%edx
0x4017d1 <fun7+13>      jg     0x4017df <fun7+27>
0x4017d3 <fun7+15>      mov    $0x0,%eax
0x4017d8 <fun7+20>      jne    0x4017ec <fun7+40>
0x4017da <fun7+22>      add    $0x8,%rsp
0x4017de <fun7+26>      ret
0x4017df <fun7+27>      mov    0x8(%rdi),%rdi
0x4017e3 <fun7+31>      call   0x4017c4 <fun7>
0x4017e8 <fun7+36>      add    %eax,%eax
0x4017ea <fun7+38>      jmp    0x4017da <fun7+22>
0x4017ec <fun7+40>      mov    0x10(%rdi),%rdi
0x4017f0 <fun7+44>      call   0x4017c4 <fun7>
0x4017f5 <fun7+49>      lea    0x1(%rax,%rax,1),%eax
0x4017f9 <fun7+53>      jmp    0x4017da <fun7+22>
```

We also noticed that the return value of fun7 must be 0, meaning the number you enter must be equal to the value stored at the root node of the tree. That's the only time no left/right traversal is needed.

We used GDB to test different numbers and see how the program worked. By checking the values in the registers, we followed the tree's path and found that 36 was the number that didn't cause the bomb to explode.



Once we confirmed that 36 was the right input through testing, we entered it, and the bomb was defused.



Final Input: "36"

After entering 36, the program didn't trigger the bomb and the Secret Phase was successfully completed.

